



Python Programming Fundamentals

String Methods

Programming Fundamentals Team

SETU

2026-09-14



Outline

1. Strings are Objects
2. Case and Whitespace Methods
3. Search and Replace
4. Split and Join
5. f-Strings
6. String Formatting Specifiers
7. String Methods in Practice



Outline

- 1. Strings are Objects**
2. Case and Whitespace Methods
3. Search and Replace
4. Split and Join
5. f-Strings
6. String Formatting Specifiers
7. String Methods in Practice



1. Strings are Objects

In Python, **everything is an object** — including strings. Objects have **methods** (functions attached to them) called with dot notation.

```
name = "hello world"
print(type(name))           # <class 'str'>

# Calling a method on a string object
print(name.upper())         # HELLO WORLD
print(name.capitalize())    # Hello world
print(name.title())         # Hello World
```



1. Strings are Objects

Strings are immutable — methods return **new** strings, they don't modify the original:

```
greeting = "hello"
upper_greeting = greeting.upper()
print(greeting)          # hello (unchanged)
print(upper_greeting)    # HELLO
```



Outline

1. Strings are Objects
- 2. Case and Whitespace Methods**
3. Search and Replace
4. Split and Join
5. f-Strings
6. String Formatting Specifiers
7. String Methods in Practice



2. Case and Whitespace Methods

```
text = "  Hello, World!  "

print(text.upper())      # "  HELLO, WORLD!  "
print(text.lower())      # "  hello, world!  "
print(text.strip())      # "Hello, World!"      removes leading/trailing
spaces
print(text.lstrip())     # "Hello, World!  "    left strip
print(text.rstrip())     # "  Hello, World!"    right strip

name = "  alice  "
clean_name = name.strip().title()
print(clean_name)        # "Alice"
```



2. Case and Whitespace Methods

```
# Checking methods – return True/False
print("PYTHON".isupper())      # True
print("python".islower())     # True
print("Python123".isalnum())  # True (letters and digits)
print("   ".isspace())        # True
```




Outline

1. Strings are Objects
2. Case and Whitespace Methods
- 3. Search and Replace**
4. Split and Join
5. f-Strings
6. String Formatting Specifiers
7. String Methods in Practice



3. Search and Replace

```
sentence = "The quick brown fox jumps over the lazy dog"

# Finding substrings
print(sentence.find("fox"))          # 16   (index of first match of "fox"-
index starts at 0)
print(sentence.find("cat"))          # -1   (not found)
print(sentence.count("the"))         # 1    (case-sensitive)
print(sentence.count("the", 0, -1, ))

# Checking start/end
print(sentence.startswith("The"))    # True
print(sentence.endswith("dog"))      # True
print(sentence.startswith("the"))    # False (case-sensitive)
```



3. Search and Replace

```
# .. contd.  
# Replacing  
new_sentence = sentence.replace("fox", "cat")  
print(new_sentence)  
# "The quick brown cat jumps over the lazy dog"  
  
# Replace only first N occurrences  
text = "aaa bbb aaa ccc aaa"  
print(text.replace("aaa", "XXX", 2))  
# "XXX bbb XXX ccc aaa"
```



Outline

1. Strings are Objects
2. Case and Whitespace Methods
3. Search and Replace
- 4. Split and Join**
5. f-Strings
6. String Formatting Specifiers
7. String Methods in Practice



4. Split and Join

split() — break a string into a list of parts:

```
csv_line = "Alice,25,Dublin,Engineer"
parts = csv_line.split(",")
print(parts)    # ['Alice', '25', 'Dublin', 'Engineer']

sentence = "the quick brown fox"
words = sentence.split()    # splits on any whitespace
print(words)    # ['the', 'quick', 'brown', 'fox']
```



4. Split and Join

join() — glue a list of strings together:

```
words = ["Python", "is", "great"]
print(" ".join(words))      # Python is great
print("-".join(words))      # Python-is-great
print("".join(words))       # Pythonisgreat
```



Outline

1. Strings are Objects
2. Case and Whitespace Methods
3. Search and Replace
4. Split and Join
- 5. f-Strings**
6. String Formatting Specifiers
7. String Methods in Practice



5. f-Strings

f-strings (formatted string literals) embed expressions directly in strings.

```
name = "Alice"
age = 25
score = 87.654

# Basic embedding
print(f"Name: {name}, Age: {age}").          # Name: Alice, Age: 25

# Expressions inside {}
print(f"Next year I'll be {age + 1}")        # Next year I'll be 26
```




5. f-Strings

```
#contd.  
# Number formatting  
print(f"Score: {score:.2f}")      # 87.65  
print(f"Score: {score:.0f}")      # 88  
print(f"Score: {score:8.2f}")     # "    87.65" (width 8)  
  
# Alignment  
print(f"{'Left':<10}|")           # Left      |  
print(f"{'Right':>10}|")          #          Right |  
print(f"{'Centre':^10}|")         #    Centre    |
```



Outline

1. Strings are Objects
2. Case and Whitespace Methods
3. Search and Replace
4. Split and Join
5. f-Strings
- 6. String Formatting Specifiers**
7. String Methods in Practice



6. String Formatting Specifiers

Specifier	Meaning	Example
:.2f	2 decimal places	3.14
:d	Integer	42
:,	Thousands separator	1,000,000
:%	Percentage	75.00%
:>10	Right-align width 10	hello
:<10	Left-align width 10	hello
:^10	Centre width 10	hello



6. String Formatting Specifiers

```
price = 1234567.89
print(f"Price: €{price:,.2f}")      # Price: €1,234,567.89

ratio = 0.756
print(f"Pass rate: {ratio:.1%}")    # Pass rate: 75.6%

item1 = ("Milk", 1.29)
item2 = ("Bread", 2.49)
item3 = ("Eggs", 3.99)

print(f"    {item1:<10} €{cost:>6.2f}") # "    Milk          €1.29"
print(f"    {item2:<10} €{cost:>6.2f}") # "    Bread          €2.49"
print(f"    {item3:<10} €{cost:>6.2f}") # "    Eggs           €3.99"
```



Outline

1. Strings are Objects
2. Case and Whitespace Methods
3. Search and Replace
4. Split and Join
5. f-Strings
6. String Formatting Specifiers
- 7. String Methods in Practice**



7. String Methods in Practice

```
def clean_and_validate_email(email):  
    """Clean and validate an email address."""  
    cleaned = email.strip().lower()  
  
    if "@" not in cleaned:  
        return None, "Missing @ symbol"  
    if "." not in cleaned:  
        return None, "Missing dot"  
    return cleaned, "Valid"  
  
def title_case_name(name):  
    """Clean and format a person's name."""  
    return " ".join( name.strip().split().title()) # "Alice Smith"
```



7. String Methods in Practice

```
# contd.  
# Test  
test1 = "  Mairead Meagher  "  
test2 = "SIOBHAN ROCHE"  
test3 = "David bowie"  
  
print(title_case_name(test1)). # "Mairead Meagher"  
print(title_case_name(test2)). # "Siobhan Roche"  
print(title_case_name(test3)). # "David Bowie"
```



Thanks for Watching - Any questions?

